

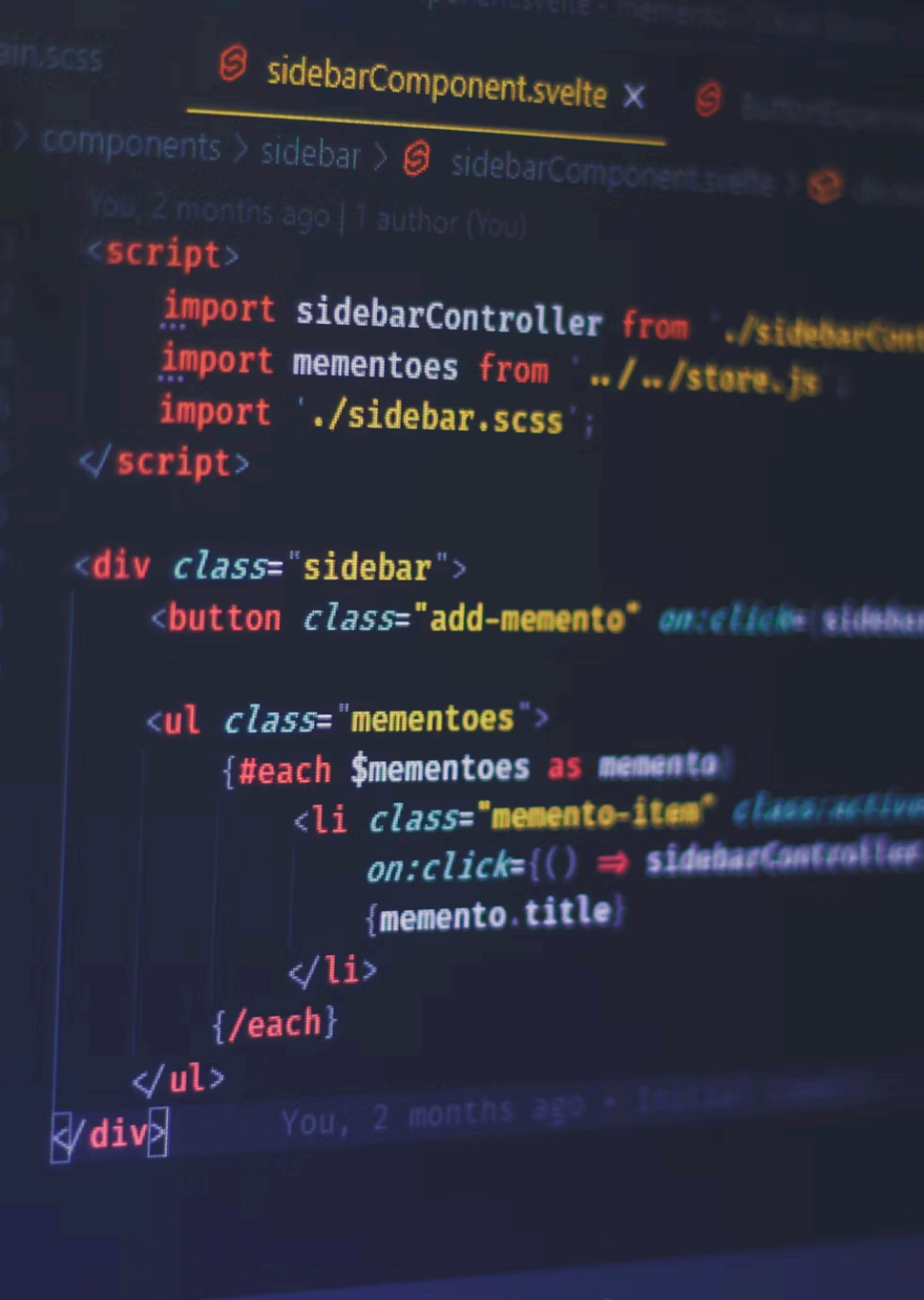


Módulo 5099: Estructuras de Control en Python

**Curso de Especialización en Programación de
Aplicaciones en Lenguaje Python**

Profesor: Jordi Pozo

IES Ribera de Castilla



¡Bienvenidos a bordo!

En este módulo vamos a aprender a darle capacidad de decisión y lógica a nuestros programas.

El objetivo de este es ser capaces de controlar el flujo de las aplicaciones.

¿Qué es el "Flujo" de un programa?

Imaginad un programa como una receta.

- Las estructuras de control son las que nos dicen: "si el agua está hirviendo, añade la pasta" o "remueve durante 5 minutos".
- Son las **instrucciones que dictan el orden y la lógica** de ejecución.

El viaje: nuestra Hoja de Ruta

El módulo tiene una duración de **40 horas**, distribuidas a lo largo de **20 semanas**.

Se ha organizado el contenido en **5 Unidades de Trabajo (UT)** para facilitar un aprendizaje progresivo y sólido.

A continuación vemos el contenido desplegado de las mismas.

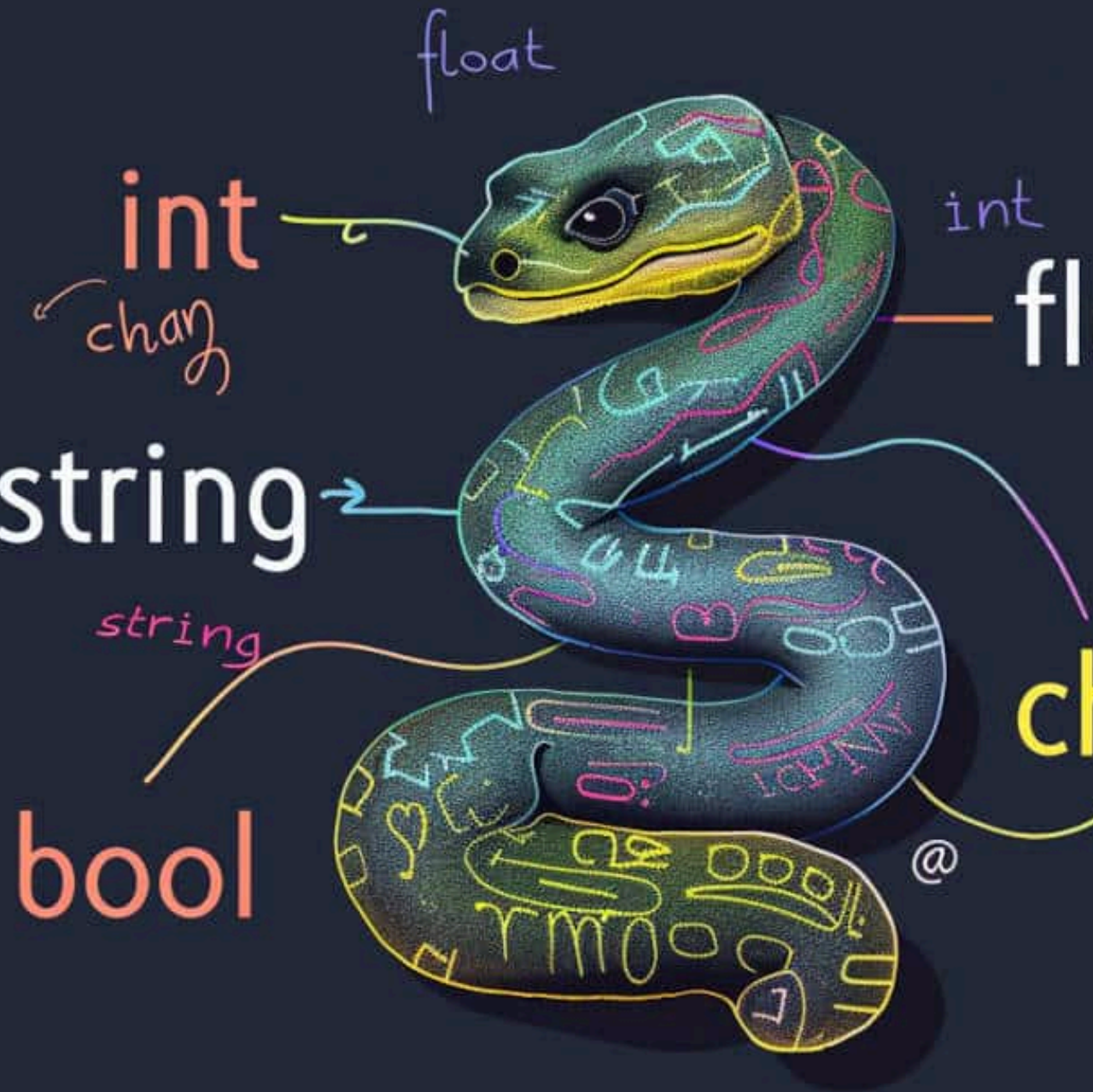
UT1: Los cimientos

(Semanas 1-4)

Empezaremos desde cero:

- Sintaxis básica de Python.
- Variables y tipos de datos.
- **Operadores y expresiones lógicas.**

Estableceremos una base fuerte para todo lo que vendrá después.



Variables: Las cajas del código

Una variable es simplemente un nombre que le damos a un espacio en la memoria para guardar información. Puede ser un número, un texto, un estado de verdadero/falso... Son los ladrillos de nuestros programas.

Expresiones lógicas: el "cerebro"

Antes de tomar una decisión, necesitamos evaluar una condición.

```
edad >= 18
```

```
contraseña == "1234"
```

Python nos dirá si estas expresiones son `True` (verdaderas) o `False` (falsas).

UT2: La toma de decisiones

(Semanas 5-8)

Aquí es donde el código empieza a "pensar". Aprenderemos a crear bifurcaciones y caminos alternativos con:

- La sentencia `if`.
- Las estructuras `if-else` y `if-elif-else`.



La sentencia `if`

Es la estructura condicional más simple. Permite ejecutar un bloque de código **únicamente si una condición es verdadera**.

Ejemplo: "Si llueve, coge el paraguas".

`if-else` & `if-elif-else`

¿Y si no llueve?

- Con `else` damos una alternativa.
- Con `elif` (else if), podemos comprobar múltiples condiciones en cadena, creando programas mucho más complejos y adaptables.

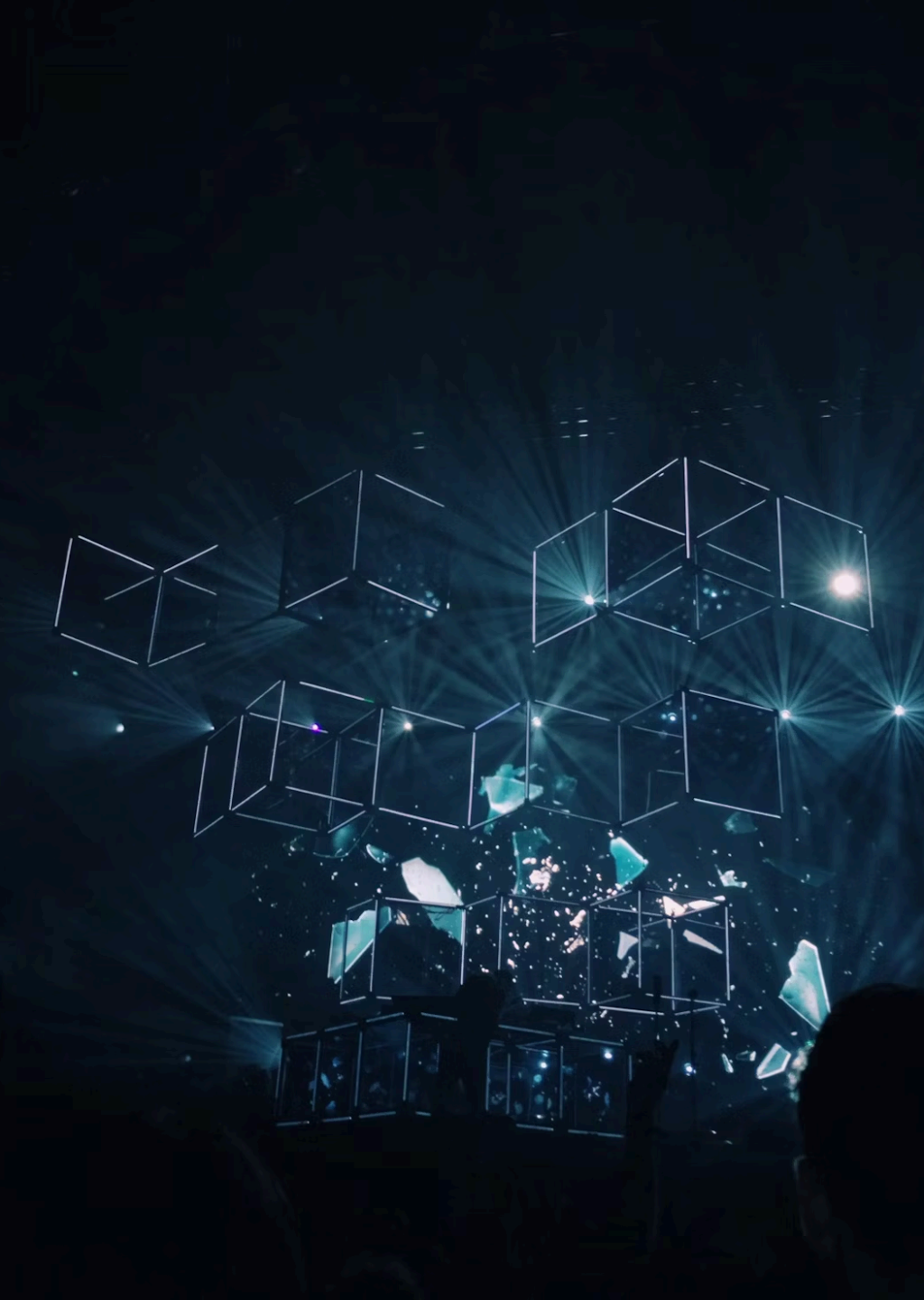
UT3: Automatizando tareas repetitivas

(Semanas 9-12)

- Aprenderemos a no repetir código.
- Los bucles son la clave para la eficiencia.
- Nos centraremos en el **bucle** `for`, el especialista en recorrer secuencias.

El bucle `for`

- Perfecto para cuando sabemos cuántas veces queremos repetir algo.
- Lo usaremos para recorrer los elementos de una lista, las letras de una palabra, o una secuencia de números generada con `range()`.

An abstract digital artwork featuring a cluster of glowing, wireframe cubes in a dark space. Bright light rays emanate from the cubes, creating a sense of depth and energy. The overall color palette is dominated by deep blues and blacks, with vibrant highlights from the cubes and light rays.

`range()` : el gran aliado

La función `range()` es fundamental para crear bucles `for` que se ejecutan un número específico de veces.

Es una herramienta potente y versátil que dominaremos a la perfección.

UT4: Bucles condicionales y control avanzado

(Semanas 13-16)

Exploraremos el **bucle** `while`, que se ejecuta *mientras* una condición sea verdadera.

También veremos cómo "modificar" el comportamiento de los bucles con `break` y `continue`.

El bucle `while`

- Ideal para situaciones donde no sabemos cuántas repeticiones necesitaremos.
Por ejemplo, un menú que se muestra hasta que el usuario elige la opción "**salir**".
- Controlaremos los bucles infinitos.

break y **continue**

Son nuestras herramientas de "*control fino*".

- **break**: Sale del bucle inmediatamente.
- **continue**: Salta a la siguiente iteración, ignorando el resto del código del bucle en esa vuelta.

UT5: ¡A construir!

(Semanas 17-20)

La última unidad está dedicada a aplicar todo lo aprendido.
Desarrollaréis un **proyecto final** donde integraréis condicionales y bucles para resolver un problema práctico.

Metodología: "Aprender haciendo"

La teoría es importante, pero la programación se aprende programando. Cada sesión tendrá una fuerte carga práctica, con ejercicios, ejemplos y retos para que podáis "ensuciaros las manos" con el código.

¡Muchas gracias!

¿Alguna pregunta?

THANK
YOU!

